

Software Engineering Project

CentraleSupélec

Hand-out: May 11, 2026

Due : 04/06/26

Programming Language: Java

1 Overview

You are required to design and implement (in Java) a software system for managing payment of bought items at a supermarket checkout. The system should compute the total bill, manage bank transactions via a POS device, and include advanced modules for **real-time inventory management** and **smart logistics**.

2 System description and requirements

You are required to develop a suitable JAVA design for the **Supermarket checkout system** based on the characteristics described in this section. The **Supermarket checkout system** consists of different components, namely the **cash register** (which concerns processing of the bought items and computation of the total bill), the **customer** (the individual that is willing to buy items from the supermarket) and the **bank payment system** (which concerns the management of the bank transaction to pay the bill). Details about the components, including a written description of the corresponding use cases are given in the following.

2.1 Core Requirements

R1: the **Supermarket checkout system** should be able to treat purchases that customers do in a supermarket

R2: a purchase consists of a set of bought items

R2b: the categories of items sold by the supermarket shall include *fruit and vegetables*, *dairy* (milk, yogurt, cheese, etc.) and *meat*

- R3:** each item is priced (the price of an item may depend on its quantity).
- R4:** the customer's payment method must include a payment by bank card (debit/credit card)
- R5:** the **Supermarket checkout system** shall provide specific discount plans that customers may adhere to. This shall include at least the following plans: *prime* which yields a 20% discount on the bought items whenever the total price of bought items is greater than or equal to 50 Euros, and *platinum* which yields a 30% discount on bought items regardless of the total price of the bought items.
- R5b:** the owner of the **Supermarket checkout system** requires to be able to introduce new types of discount plans or to modify existing ones.
- R6:** the **Supermarket checkout system** should apply specific pricing policies to dedicated categories of items: for example it may apply a 10% reduction on fresh vegetables or a 5% reduction to dairy products.
- R6b:** the owner of the **Supermarket checkout system** requires to be able to introduce new types of pricing policies for newly introduced categories of products, or to modify existing ones.
- R7:** customers of the **Supermarket checkout system** may choose to have a purchase delivered at their (home) address
- R8:** the **Supermarket checkout system** should be able to apply specific charges for home delivery: for example delivery should be charged at a cost which depends on the total weight of bought items as well as on the distance for delivering the bought items from the supermarket to the customer's delivery address. For example deliveries under 10 Kg and within 30Km distance may be charged a fixed amount (e.g. 15 Euros) whereas delivery between 10 and 50 Kg should be charged a fixed amount plus a percentage of the total price for the bought items. Deliveries of more than 50 Kg are not supported and should be refused by the system.
- R8b:** the **Supermarket checkout system** should apply specific discount policies for deliveries depending on the plans a customer adheres to: e.g. *prime* customers are charged only 50% of the delivery cost, whereas delivery for *platinum* customers is for free.

2.2 Advanced System Extensions

R9: Inventory Management & Threshold Alerts The system shall maintain a live stock count. Every finalized sale must decrement the inventory. Using the **Observer Pattern**, the system must notify a "Manager" or "Supplier" module whenever perishable items (e.g., *meat* or *dairy*) fall below a configurable threshold.

R10: Dynamic Logistics & Time-Slots Home delivery (R7) is extended to include 2-hour delivery windows. The system must prevent overbooking based on available vehicle capacity and apply **Dynamic Pricing**: higher fees for peak hours and discounts for "eco-friendly" slots where a delivery truck is already in the customer's vicinity.

2.3 Customer

A Customer is characterised by

- **first name**: the customer's first name
- **surname**: the customer's surname
- **numerical ID**: a unique numerical ID that characterises the customer
- **customer plan**: the type of registration that characterises the customer. The system should support (at least) the following type of plans
 - **normal**: no registration fees, no discount
 - **prime**: 50 Euro annual fee, 20% discount on the bill (applied only when the total of bought items is greater than or equal to 50 Euros, see R5)
 - **platinum**: 200 Euro annual fee, 30% discount on the bill (applied regardless of the total of bought items, see R5)

2.4 Supermarket cash register scenario

The cash register is in charge of computing the bill for the items a customer wishes to buy. The bill should be computed based on the price of each item and taking into account both the customer discount plan (if any) as well as the current discount policy that the supermarket may apply to items of a specific family (e.g. a price reduction on all fruits and vegetables, or on dairy products, etc.)

The functioning of the cash register can be described by the following course of actions:

- A customer arrives at the checkout with items to pay for.
- The cashier records the identification number of each item, as well as the quantity if it is greater than one.

- The checkout displays the price of each item and its label.
- When all items are registered, the cashier signals the end of the sale.
- The cash register displays the total price for the bought items (the total price is computed by taking into account the shop's pricing policy as well as the customer's registration plan).
- The customer inserts his credit card in the point of sale (POS) terminal (which is part of the cash register): the POS sends an authorisation request to a payment authorisation center
- The cash register registers the sale and prints a receipt.
- The cashier gives the receipt to the customer.

2.5 Bank Point Of Sale scenario

The **Supermarket checkout system** system is equipped with a POS device that allows customers to pay for the bought items through a credit/debit card. The POS is in charge of managing the Bank transaction (for the amount corresponding to the bill of the bought items) by reading the card, asking for the PIN code, checking the code is correct, checking with the bank if the customer has sufficient credit and then validating/refusing the payment.

The functioning of the POS can be described by the following course of actions:

- the Customer inserts the card in the POS
- the POS device verifies the card is indeed a bank card
- the POS prompts the Customer to enter the PIN
- the Customer enters the PIN
- the POS verifies that the entered PIN matches that stored on the card chip
- the POS asks authorisation to proceed with debiting the transaction on the Customer's account to the Bank's authorisation system
- the Bank's authorisation system authorises the POS for the transaction
- the POS asks the Customer if he/she wants a receipt of the transaction
- the Customer asks for a receipt
- the POS prints the receipt
- the Customer takes back the card and the receipt

2.6 Transaction authorisation systems

The bank's transaction authorisation system (TAS) is in charge of offering the following services:

- open a secure connection with a *registered user of the TAS*
- issue an authorisation to perform a transaction to a *registered user of the TAS* concerning a given *Client of the bank* and for a given amount.
- close a previously opened secure connection with a *registered user of the TAS*

The functioning of the TAS can be described by the following course of actions:

- the TAS receives a secure connection request by an online client (the POS)
- the TAS opens the secure connection with the requesting client
- the client (POS) sends transaction details (card number and total amount to be charged) to the TAS
- the TAS checks if the transaction can be authorised (total amount is available on card holder's account)
- the TAS issues the authorisation to the requesting client (POS) and performs the transaction (debiting the card holder's account and crediting the shop account corresponding to the POS)
- the TAS closes the secure connection established with the shop

3 Part 2: Supermarket checkout system user interface

The part 2 of the project is about realising a user interface for the **Supermarket checkout system**-app. The user interface consists a command-line user interface (CLUI). For insights about CLUI see https://en.wikipedia.org/wiki/Command-line_interface and some examples in Java

<https://dzone.com/articles/java-command-line-interfaces-part-29-do-it-yourself>

You can start off from a **toy example of CLUI written in Java and whose code is available on Edunao**. https://centralesupelec.edunao.com/pluginfile.php/526474/mod_resource/content/1/CLI_toy.java

3.1 Supermarket checkout system command line user interface

The command line interpreter provides the user with a (linux-style) terminal like environment to enter commands to interact with the **Supermarket checkout system** core. A command consists of the **command-name** followed by a blank-separated list of (string) arguments:

`command-name <arg1> <arg2> ... <argN>`

a command without argument is denoted `command-name <>`.

In particular the command-line interpreter should feature the following list of commands:

- `login <username> <password>`: to allow a user to perform the login (**remark**: a **Supermarket checkout system** manager user with username: `ceo` and password: `123456789` is assumed to exist)
- `logout <>`: to allow the currently logged on user to log off
- `setup <>`: to load the default configuration of the **Supermarket checkout system** (default catalogue with the three mandatory item categories of R2b, one cashier, one customer, default discount plans *prime/platinum* and pre-registered test bank cards)
- `registerCashier <firstName> <lastName> <username> <password>`: for the currently logged on manager to add a cashier to the system
- `registerCustomer <firstName> <lastName> <username> <address> <password>`: for the currently logged on manager to add a customer (by default each new customer has the **normal** plan and an auto-generated test bank card)
- `addItem <itemName> <categoryName> <unitPrice> <weight> <initialStock>`: for the currently logged on manager to add a sellable item to the catalogue (if `<categoryName>` is unknown, the category is created on the fly – this covers R6b)
- `restock <itemName> <quantity>`: for the currently logged on manager to replenish the stock of an item
- `setCategoryDiscount <categoryName> <discountPercent>`: for the currently logged on manager to apply a category-level pricing policy (R6), e.g. `setCategoryDiscount fruit-and-vegetables 10` for a 10% reduction
- `subscribeToPlan <planName>`: for the currently logged on customer to subscribe to a discount plan (**prime**, **platinum**, or any other plan defined at setup) – the annual fee is charged immediately
- `startCheckout <customerUsername>`: for the currently logged on cashier to open a checkout session for the given customer (loads the customer's plan so it can be applied to the bill)

- **scanItem** <itemName> <quantity>: for the currently logged on cashier to scan a given quantity of an item in the current cart
- **computeBill** <>: for the currently logged on cashier to display the total bill of the current cart, applying both the customer's discount plan (R5) and the active category-level pricing policies (R6), with delivery fees included if a delivery has been requested
- **requestDelivery** <address>: for the currently logged on customer to register a pending delivery request for the next purchase; the request is applied to the cart of the next **startCheckout** opened for that customer. The delivery fee is computed from the total weight and the distance, and is reduced or waived according to the customer's plan (R8b).
- **pay** <cardNumber> <pin>: for the currently logged on cashier to launch the POS payment for the current bill – the POS contacts the TAS which validates PIN, checks balance and either authorises or refuses; on success the inventory is decremented (possibly triggering the R9 low-stock notification) and a receipt is printed
- **simulatePayment** <outcome>: for the currently logged on cashier to force the outcome of the next **pay** call – <outcome> in {SUCCESS, INSUFFICIENT_FUNDS, PIN_WRONG, AUTH_DENIED}; useful to obtain reproducible unhappy-path scenarios without manipulating bank state
- **showInventory** <>: for the currently logged on manager to display the catalogue with current stock levels and a marker for items below their low-stock threshold (R9)
- **showRevenue** <>: for the currently logged on manager to display the total revenue of the **Supermarket checkout system** since creation
- **runTest** <testScenario-file>: for a generic user of the CLUI (no need to login) to execute the list of CLUI commands contained in the file passed as argument
- **help** <>: for a generic user of the CLUI (no need to login) to display the list of available CLUI commands

Error messages and CLUI. The CLUI must handle all possible types of errors, i.e. syntax errors while typing in a command, and misuse errors – for example trying to scan an item which is not in the catalogue, restocking an unknown item, paying with an unregistered bank card, starting a checkout for a customer that does not exist, etc.

4 Project testing (mandatory)

In order to evaluate your implementations we (the Testers of your project) require you (the Developers) to equip your projects with both standard **JUnit tests** (for each class) and a **test scenario**, described below. Both JUnit tests and the test scenario are mandatory parts of your project realisations, as we (the Testers) will resort to both of them to test your implementations.

4.1 Junit tests

Each class in your project must contain JUnit tests for the most significant methods (i.e. excluded *getters* and *setters*).

Hint: if you follow a Test Driven Development approach you will end up naturally having all JUnit tests for all of your classes.

4.2 Test scenario

In order to test your solution you are required to include in the project

- one initial configuration file (called `my_supermarket.ini`), automatically loaded at starting of the system,
- at least one test-scenario file (called `testScenario1.txt`).

Configuration file. An initial configuration file must ensure that, at startup (after loading this file) the system contains at least the “standard” setup corresponding to the default version of the CLUI command `setup`.

Test-scenario file. A test-scenario file contains a number of CLUI commands whose execution allows for reproducing a given test scenario, typically setting up a given configuration of the `mysupermarket` system (e.g. creation of users, addition of items to the catalogue, registration of a customer to a discount plan, simulation of a checkout with successful or failed payment, triggering of a low-stock notification, computation of statistics over customers and items, etc.). Lines beginning with the `#` character are treated as comments and are ignored by the `runTest` command. You may include several test-scenario files (e.g. `testScenario1.txt`, `testScenario2.txt`, ...). For each test-scenario file you provide us with you **MUST** include a description of its content (what does it test?) in the report. We are going to run each test-scenario file through the `runTest` command of the CLUI (see CLUI commands above):

```
runTest testScenario1.txt
```


5 Deliverables

You are required to deliver the link to the github/gitlab repository containing the project's material. This repository should contain:

- the Java code for your implementation of the **Supermarket checkout system**.
- a PDF document containing the report of the project

5.1 Report

The report should contain the following informations:

- description of design decisions (what impacts the list of requirements had on your code design?)
- details about the design patterns you used: you should clearly describe which pattern you used to solve which problem
- design analysis: advantages/limitations of your solution
- UML diagrams: the report should contain (mandatorily) class diagrams of your implementation. Use case and sequence diagrams should also be included.
- **MANDATORY: the test scenario description**
- **MANDATORY: how to test your realisation** : describe the test code you have tested your implementation with and describe how to easily run the test code (it should be possible to easily reproduce your tests).
- how the workload has been split and its realisation (who did what in a commented table with mandatory columns `design` | `code` | `UML diagrams` | `JUnit test` and lines class or task)

The quality of the report is an important aspect of the project's mark, thus it is warmly recommended to write a quality report.